



SPLoRA: Sonnet Generation and Paraphrase Detection with LoRA

Maya Vendhan¹, Joseph Bailey¹

¹Department of Computer Science, Stanford University

Stanford
Computer Science

Motivation & Central Question

Large pretrained language models achieve strong performance across NLP tasks, but adapting them typically requires fine-tuning hundreds of millions of parameters. **Low-Rank Adaptation (LoRA)** addresses this by learning low-rank updates to weight matrices while keeping original parameters frozen.

Central question: Does a single LoRA configuration perform well across fundamentally different tasks, or must adapter design choices be tuned per-task?

We evaluate LoRA on two distinct tasks:

- Paraphrase detection (QQP)** — a structured classification task requiring subtle semantic distinctions
- Shakespearean sonnet generation** — a creative generative task requiring stylistic and structural properties

We systematically study adapter placement, rank, scaling factor, dropout, and learning rate to understand how parameter-efficient adaptation behaves under different learning objectives.

Approach

Following Hu et al. (2022) [1], the adapted weight matrix is:

$$W = W_0 + \frac{\alpha}{r}BA,$$

where W_0 is the frozen pretrained weight, $A \in \mathbb{R}^{r \times d_1}$ and $B \in \mathbb{R}^{d_2 \times r}$ are trainable low-rank matrices with rank $r \ll d$, and α is a scaling factor A is initialized with random Gaussian, B with zeros.

Adaptation strategies evaluated:

- Attention-only:** query + value (qv), or all attention projections (qvko)
- MLP-only:** feed-forward layers
- MLP + Attention:** both modules simultaneously
- Full LoRA:** attention + MLP + embeddings

Experimental Setup

Model: GPT-2 (124M parameters) with LoRA applied directly to transformer layers.

Hyperparameters explored:

- Rank $r \in \{4, 8, 16, 32\}$
- Scaling factor $\alpha \in \{4, 8, 16, 32\}$
- Dropout $p \in [0, 0.3]$
- Learning rate: 5×10^{-6} to 10^{-4}
- Up to 20 epochs with early stopping

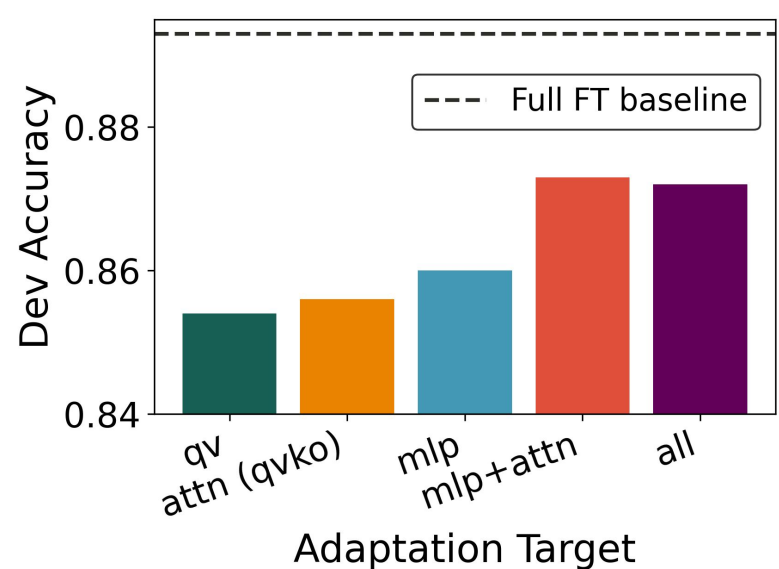
Metrics: Classification accuracy (QQP); chrF score (sonnets).

Baseline: We fine-tune all 124M GPT-2 parameters separately for each task.

Results: Paraphrase Detection

Best LoRA achieves 0.888 dev accuracy - within 0.6% of full fine-tuning (0.893) - while training only ~1% of parameters (1.33M vs 124M). Test leaderboard score: 0.873.

QQP Dev Accuracy by Adapter Placement ($r=4, \alpha=4, lr=10^{-5}, \text{dropout}=0$)



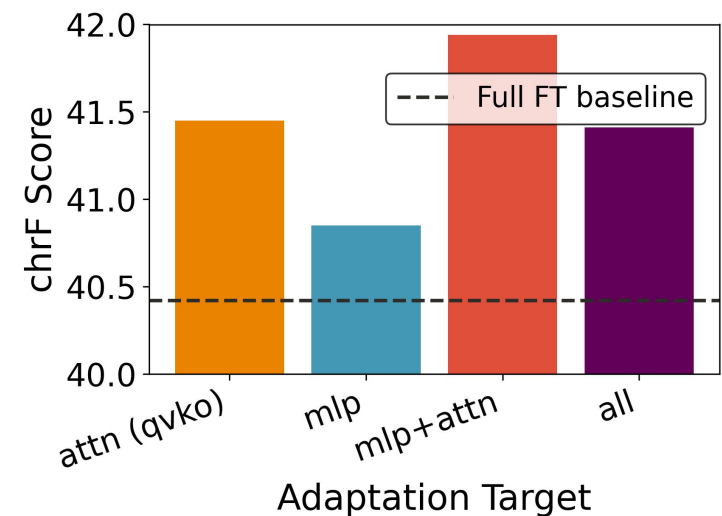
Selected Hyperparameter Comparisons ($lr=10^{-5}$)

Target	r	α	p	Dev Acc
qvko	8	16	0.05	0.877
mlp	8	16	0	0.881
mlp	8	16	0.05	0.882
mlp+attn	8	16	0	0.889
mlp+attn	8	16	0.05	0.887

Results: Sonnet Generation

Best LoRA achieves 41.94 dev chrF - exceeds baseline of full fine-tuning (40.417), while training only ~1% of parameters. Test leaderboard score: 40.637.

Dev chrF by Adapter Placement ($r=8, \alpha=32, lr=2 \times 10^{-5}, \text{dropout}=0.2$)



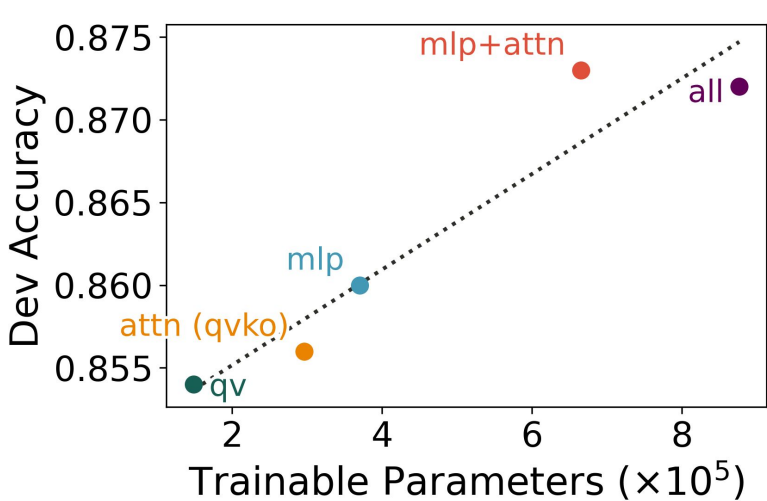
Alpha Sweep (mlp+attn, $r=8, lr=10^{-5}, p=0.1$)

α	chrF
4	38.14
8	39.25
16	40.53
32	41.43

Dropout Sweep (mlp+attn, $r=8, \alpha=32, lr=2 \times 10^{-5}$)

Dropout p	chrF
0.10	41.50
0.20	41.94
0.25	41.52
0.30	41.38

Parameter efficiency of LoRA configurations on QQP



Optimal Configurations by Task

Hyperparameter	Classification	Generation
Adapter target	mlp+attn	mlp+attn
Rank r	8	8
Scaling α	16	32
Learning rate	10^{-5}	2×10^{-5}
Dropout p	0	0.2

Analysis & Key Findings

Adapter Placement:

Adapting both MLP and attention layers consistently outperforms single-module configurations across both tasks. MLP layers contribute substantially to task-specific representations. Embedding adaptation adds parameter cost without meaningful benefit.

Rank vs. Coverage:

Increasing rank from $r=4$ to $r=8$ improves performance, but higher ranks show diminishing returns when adapters are applied broadly. Broader module coverage reduces the effective dimensionality needed per adapter — consistent with QLoRA findings that coverage can compensate for lower rank [2].

Scaling Factor α :

Classification favors moderate scaling ($\alpha=16$), while generation benefits from stronger scaling ($\alpha=32$). Classification requires subtle adjustments to existing representations; generation needs stronger updates to capture stylistic features.

Dropout Effects:

Classification shows no consistent benefit from dropout (best config: $p=0$). Generation benefits from moderate dropout ($p=0.2$), suggesting regularization helps prevent overfitting on the small sonnet corpus.

1% of Parameters, 100% of Performance

LoRA trains ~1.33M parameters (~1% of GPT-2's 124M):

- QQP:** 0.888 accuracy (within 0.6% of full fine-tuning at 0.893)
- Sonnets:** 41.94 chrF (surpasses full fine-tuning at 40.417)

Constraining updates to a low-rank subspace can act as a beneficial regularizer for generative tasks, preventing overfitting to small training corpora.

Conclusions & Future Research

Conclusions:

- Optimal configurations differ across tasks, so don't rely on a single default LoRA recipe.
- For our tasks, adapting MLP+attention layers with rank $r=8$ works best.

Future Research:

- Future work should run multiple random seeds for each configuration to measure variance.
- Future work should use better generation evaluation than chrF to better measure poetic quality.

References:

- [1] Hu et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.
[2] Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS.